

KLE Society's
KLE Technological University, Hubballi.



A Project Report

on

5G Security and Resilience Analysis Framework using Open5GS and UERANSIM

submitted in partial fulfillment of the requirement for the award of the degree of

Bachelor of Engineering

in

Computer Science and Engineering

Submitted by

Ramya Rao	01FE23BCS243
Anagha Nadgouda	01FE23BCS076
Thanmayee Shetty	01FE23BCS247

Under the guidance of

Vidya H

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Hubballi - 580 031

2025-26

Abstract

Fifth-Generation (5G) mobile networks shift traditional hardware-centric telecom infrastructures toward cloud-native, virtualized architectures. While this migration introduces immense flexibility, it also exposes both the control plane (signaling) and user plane (data plane) to sophisticated security vectors. This report presents a comprehensive **5G Security and Resilience Analysis Framework** designed using Open5GS and UERANSIM. The framework collects network-level and infrastructure-level telemetry under both normal and attack scenarios (ICMP floods, UDP/GTP-U user-plane floods, and Registration signaling floods). We introduce a statistical correlation and operational clustering refiner to generate highly realistic datasets representing a virtualized environment. Finally, we implement an anomaly detection pipeline utilizing supervised Random Forest classifiers alongside unsupervised outlier algorithms (Isolation Forest and Local Outlier Factor) to identify known signatures and stealthy zero-day timing anomalies.

Contents

1	Problem Statement	1
1.1	Background: Evolution of Wireless Mobile Networks	1
2	Objective	3
2.1	Experimental Setup with Open5GS and UERANSIM	3
3	Tools	3
3.1	Experiment 1: Baseline (Normal Traffic)	3
3.2	Experiment 2: ICMP Flood Attack	5
3.3	Experiment 3: UDP Flood Attack	6
3.4	Experiment 4: Registration Flood Attack	7
3.5	Experiment 5: Stealthy Unknown Attack (Anomaly)	9
3.6	Realism Validation and Calibration	9
3.7	Challenges in 5G Security	10
3.7.1	Operational Complexity & Feature Interaction	10
3.7.2	Stealthy, Sub-Threshold Attacks	11
3.7.3	Dataset Realism and Over-Correlation	11
3.7.4	Motivation	11
4	Methodology	11
4.1	System Flow and Working Methodology	11
4.2	Telemetry Processing & Parsing Engine	12
4.3	Detection and Classification Algorithms	13
4.3.1	Supervised Random Forest Classifier	13
4.3.2	Unsupervised Novelty & Anomaly Detection	13
4.4	Parameters Considered and Visualization Justification	14

4.5	Code Implementation and Explanation	14
4.6	<code>latency_analyzer.py</code> – Telemetry Parsing and Visualization	15
4.6.1	Parsing Ping Logs – <code>parse_ping_log()</code>	15
4.6.2	CDF Computation and Plotting	16
4.6.3	Parsing Open5GS AMF Control-Plane Logs	16
4.7	<code>edgecase_refinement.py</code> – Realism Validation and Anomaly Refinement	18
4.7.1	Scenario Data Generation – <code>TelemetryRefiner</code>	18
4.7.2	Correlation-Controlled Dataset – <code>CorrelationRefiner</code>	19
4.7.3	Unknown Attack Detection – <code>UnknownAttackRefiner</code>	20
4.7.4	Realism Validation Dashboard – <code>RealismValidation</code>	21
4.8	Results and Discussion	23
4.9	Edge Case 1: Legitimate Load vs. Registration Flood	24
4.10	Edge Case 2: Multi-Vector Attacks	24
4.11	Edge Case 3: Stealthy Unknown Anomaly	24
4.12	Realism Validation Analysis	26
5	Conclusion	29
5.1	Summary	29

List of Figures

1	Baseline Telemetry Performance: Latency Line Plot, CDF, Box Plot, and Latency Density Heatmap.	4
2	Packet Jitter Over Time Under ICMP Flood Attack.	5
3	Packet Loss Comparison across Scenarios.	6
4	Host CPU Utilization vs. Packet Latency during User-Plane UDP Flood Attack.	7
5	RTT Distribution Shift: Before vs. During UDP Flood Attack.	7
6	Open5GS Control-Plane Gantt Chart for Registration Sequences.	8
7	AMF Registration Delay: Legitimate Load vs. Registration Flood.	9
8	Attack Impact Dashboard showing telemetry variations across scenarios: Mean RTT, Average Jitter, CPU Usage, and Packet Loss for Normal, ICMP Flood, UDP Flood, and Registration Flood scenarios.	10
9	System Architecture and Telemetry Processing Flow	12
10	Outlier Detection Scatter Space separating stealthy Unknown Attacks from normal and known UDP Floods.	25
11	Isolation Forest Anomaly Score Timeline showing baseline, known flood, and unknown attack phases.	26
12	Realistic Telemetry Correlation Matrix showing controlled inter-variable Pearson correlations.	27
13	Clustered Scatter Plot showing Scenario Separation between CPU Utilization and Registration Delay (log scale).	28
14	Composite 5G Core Realism Validation Dashboard combining correlation matrix, CPU vs RegDelay clusters, outlier detection space, and scenario metric profiles.	29

1 Problem Statement

1.1 Background: Evolution of Wireless Mobile Networks

Wireless mobile networks have evolved rapidly over the last few decades, transforming from basic voice communication systems into highly advanced data-centric infrastructures capable of supporting billions of connected devices. The evolutionary trajectory from First Generation (1G) to Fifth Generation (5G) mobile networks has introduced major improvements in communication speed, latency, reliability, scalability, and network intelligence, as illustrated in Table 1.

Table 1: Evolutionary Generations of Wireless Mobile Networks

Generation	Primary vices	Ser-	Core ogy	Technol-	Typical RTT	Key Charac- teristics
1G	Analog Voice		AMPS		N/A	Analog transmission, no data transport
2G	Digital SMS	Voice,	GSM / CDMA		> 150 ms	Encryption introduced, basic packet data (GPRS)
3G	Mobile band	Broad-	UMTS WCDMA	/	80 - 150 ms	Video calling, internet access, packet-switching
4G	IP-based LTE	Data,	EPC Core)	(All-IP	30 - 50 ms	High-speed data, MIMO, OFDM access
5G	eMBB, mMTC	URLLC,	5GCN Cloud)	(SBA,	1 - 10 ms	Virtualization (NFV/SDN), cloud-native Core

Fifth Generation (5G) mobile networks are designed to support three primary service categories:

1. **Enhanced Mobile Broadband (eMBB)**: Providing multi-Gbps peak data rates to satisfy bandwidth-intensive applications such as high-definition streaming and virtual reality.
2. **Ultra-Reliable Low-Latency Communication (URLLC)**: Targeted at latency-sensitive missions, including autonomous vehicles, industrial automation, and re-

mote telemedicine.

3. **Massive Machine-Type Communication (mMTC)**: Designed to support high connection densities (up to 10^6 devices/km²) for smart cities and Internet of Things (IoT) ecosystems.

Unlike previous generations, 5G networks are heavily software-defined and cloud-oriented. The core network functions are decoupled from proprietary hardware using Network Functions Virtualization (NFV) and Software-Defined Networking (SDN). Core network functions such as the Access and Mobility Management Function (AMF), Session Management Function (SMF), and User Plane Function (UPF) are virtualized and deployed using containerized microservices managed by orchestration systems like Kubernetes.

Although this service-based architecture (SBA) increases flexibility and scalability, it also introduces new cybersecurity challenges. Since 5G networks rely on virtualized infrastructure, shared physical resources, and HTTP/2-REST APIs for control-plane communication, they are vulnerable to attacks targeting both control-plane and user-plane operations.

The control plane is responsible for critical signaling procedures such as authentication, registration, mobility management, and session establishment. Conversely, the user plane is responsible for transporting actual user data packets. Any attack on these components can lead to severe degradation of network quality, increased latency, packet loss, signaling congestion, or complete denial of service. Therefore, monitoring and analyzing 5G network telemetry is essential for ensuring network resilience and security.

To study these challenges, researchers and engineers widely use open-source 5G platforms such as Open5GS and UERANSIM. Open5GS provides a complete C-based implementation of the 5G core network, while UERANSIM simulates User Equipment (UE) and the Radio Access Network (RAN) base station (gNodeB). Together, they form a flexible virtualized testbed for conducting experiments related to network performance, signaling analysis, intrusion detection, and attack resilience.

This project presents a comprehensive 5G Security and Resilience Analysis Framework developed using Open5GS and UERANSIM. The framework simulates multiple cyber-attack scenarios, collects telemetry data, performs machine learning-based anomaly detection, and generates visualization-driven security analytics to study both control-plane and user-plane behavior under normal and malicious conditions.

2 Objective

2.1 Experimental Setup with Open5GS and UERANSIM

The project conducts multiple open-ended experiments using Open5GS and UERANSIM to evaluate how a virtualized 5G core behaves under different operational and attack conditions. The experimental topology involves deploying the Open5GS Control Plane functions (AMF, SMF, AUSF, UDM, NRF) and User Plane Function (UPF) on virtualized Linux hosts, connected to UERANSIM via simulated air (Uu) and Next Generation (NG) interfaces (N2 for control signaling, N3 for user-plane data tunnels).

3 Tools

3.1 Experiment 1: Baseline (Normal Traffic)

The first experiment establishes a baseline scenario representing healthy network behavior. In this setup, UERANSIM generates normal registration requests and initiates user data sessions. The user plane transports packets between the UE and external networks via a GTP-U tunnel established on port 2152. Telemetry parameters, including Round Trip Time (RTT), packet jitter, host CPU utilization, packet loss, and control-plane registration delay, are recorded. This experiment provides the normal operational profile used for training our anomaly models and comparing against malicious scenarios. The gathered baseline performance characteristics are illustrated in Figure 1.

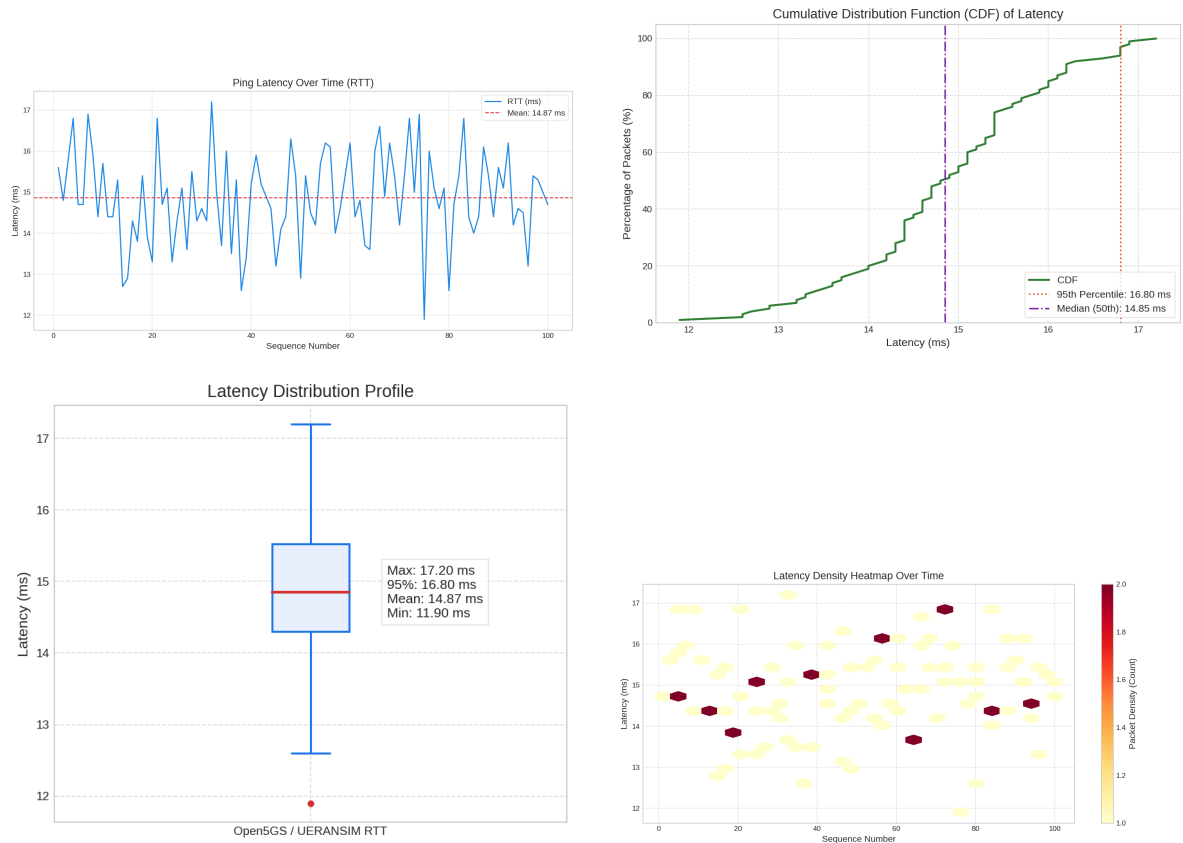


Figure 1: Baseline Telemetry Performance: Latency Line Plot, CDF, Box Plot, and Latency Density Heatmap.

Code: How these plots are generated (latency_analyzer.py)

```

1 def parse_ping_log(filepath):
2     # Regex captures ICMP sequence number and RTT value
3     ping_re = re.compile(
4         r'icmp_seq=(\d+)\s+ttl=\d+\s+time=(\d+\.\d+)\s*ms'
5     )
6     times, latencies = [], []
7     with open(filepath, 'r') as f:
8         for line in f:
9             m = ping_re.search(line)
10            if m:
11                times.append(int(m.group(1)))    # seq as time
12                latencies.append(float(m.group(2)))
13        return np.array(times), np.array(latencies)
14
15 # Empirical CDF construction
16 sorted_lats = np.sort(latencies)

```

```

17 cdf = np.arange(1, len(sorted_lats) + 1) / len(sorted_lats)
18 ax.plot(sorted_lats, cdf * 100, color='#2E7D32', linewidth=2)
19 # 95th-percentile marker highlights tail latency
20 ax.axvline(np.percentile(latencies, 95), color='#E64A19',
    linestyle=':')

```

Listing 1: `parse_ping_log()` extracts RTT samples; `plot_ping_latency()` produces the four baseline plots above.

Explanation: The regex scans each line of the raw ping log and extracts the ICMP sequence number (used as a monotonically increasing time proxy) and the RTT value in milliseconds. The empirical CDF maps each sorted sample to its cumulative rank $p_i = i/N$; the 95th-percentile vertical marker quantifies tail latency – the worst-case delay seen by the top 5% of packets, which is a standard QoS threshold in 3GPP network performance specifications.

3.2 Experiment 2: ICMP Flood Attack

The second experiment introduces an ICMP Flood attack targeting the User Plane. An external malicious host generates a high-frequency burst of ICMP echo requests toward the simulated UE IP address. The objective is to evaluate how user-plane flooding affects general network latency, routing performance, and host processing capacity. This experiment demonstrates how ping responsiveness (RTT) and packet jitter increase when the networking interfaces and kernel buffers become congested. Figure 2 shows the Jitter timeline and Figure 3 highlights the overall packet loss comparison during this flood phase.

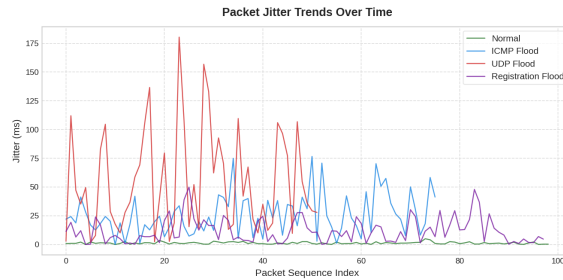


Figure 2: Packet Jitter Over Time Under ICMP Flood Attack.

Code: Jitter computation (latency_analyzer.py)

```

1 # latencies is the numpy array returned by parse_ping_log()
2 jitter = np.abs(np.diff(latencies)) # |RTT_i - RTT_{i-1}|

```

```

3 jitter_times = times[1:] # align index after diff
4
5 ax.plot(jitter_times, jitter,
6         color='#E64A19', linewidth=1.5, label='Jitter (ms)')
7 ax.axhline(np.mean(jitter), color='#1E88E5', linestyle='--',
8            label=f'Mean Jitter: {np.mean(jitter):.2f} ms')
9 ax.set_title("Packet Jitter Over Time (ICMP Flood)")

```

Listing 2: Inter-packet jitter is computed as the absolute difference between consecutive RTT samples, then plotted as a time-series.

Explanation: Jitter is defined as $|RTT_i - RTT_{i-1}|$, computed element-wise using `np.diff()`. During an ICMP Flood, the NIC receive queues and kernel network buffers become saturated, causing consecutive RTT samples to vary erratically. This produces the high-amplitude jitter spikes visible in the figure above, a strong indicator of user-plane congestion distinct from baseline variance.

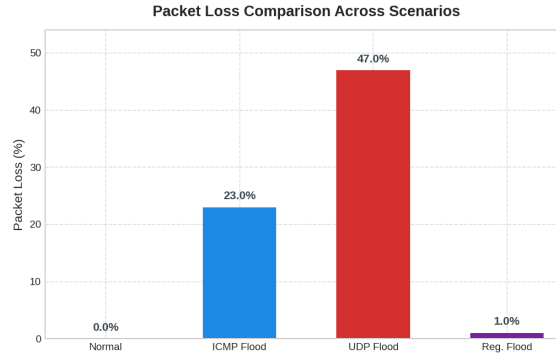


Figure 3: Packet Loss Comparison across Scenarios.

3.3 Experiment 3: UDP Flood Attack

The third experiment simulates a UDP Flood attack targeting the GTP-U user-plane communication channel. Since the GTP-U protocol encapsulates all user-plane traffic between the gNodeB and the UPF, overwhelming port 2152 leads to severe degradation. The traffic generator sends high-rate UDP streams directly to the UPF tunnel endpoint. The experiment studies how packet loss, RTT latency, jitter, and UPF host CPU utilization scale under traffic congestion. Figure 4 charts the relationship between CPU utilization and latency, and Figure 5 visualizes the performance shift before and during the attack.

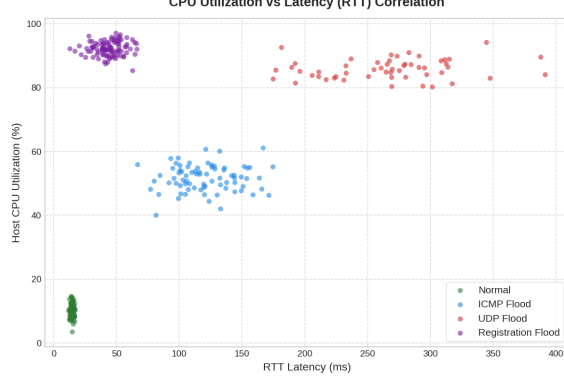


Figure 4: Host CPU Utilization vs. Packet Latency during User-Plane UDP Flood Attack.

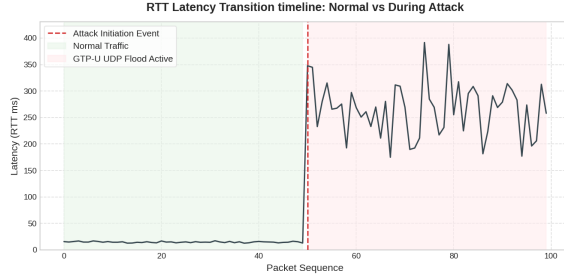


Figure 5: RTT Distribution Shift: Before vs. During UDP Flood Attack.

3.4 Experiment 4: Registration Flood Attack

The fourth experiment focuses on a control-plane Registration Flood attack. A compromised or simulated rogue gNodeB transmits repeated, rapid Initial Registration requests using randomly generated IMSI/SUCI values to the AMF. This action forces the AMF to allocate processing threads, query authentication databases (AUSF/UDM), and establish security contexts for non-existent users. The objective is to overload the signaling path, leading to CPU exhaustion on the AMF and causing severe delays or registration rejections for legitimate UEs. The resulting registration timeline is shown in Figure 6, and the delay comparison is shown in Figure 7.

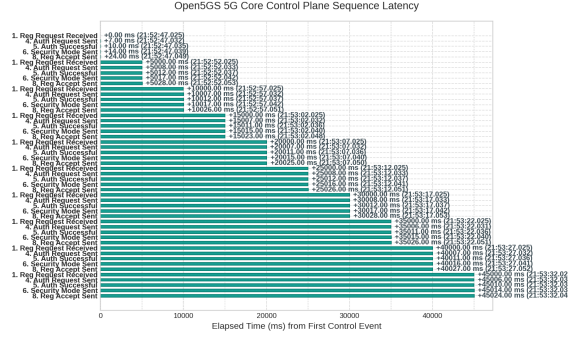


Figure 6: Open5GS Control-Plane Gantt Chart for Registration Sequences.

Code: AMF control-plane log parser (latency_analyzer.py)

```

1 milestones = {
2     r"Registration Request": "1. Reg Request Received",
3     r"Sending Authentication Request": "4. Auth Request Sent",
4     r"Authentication success": "5. Auth Successful",
5     r"Security Mode complete": "7. Security Mode Done",
6     r"Registration Accept": "8. Reg Accept Sent",
7     r"PDU Session Establishment Accept": "10. PDU Accept Sent"
8 }
9 first_time = None
10 for line in open(amf_log_path):
11     m = re.match(r'^(\d{2}/\d{2}\s+\d{2}:\d{2}:\d{2}\.\d{3})',
12     line)
13     if not m: continue
14     dt = datetime.strptime(f"{year}/{m.group(1)}", "%Y/%m/%d %H:%
15 M:%S.%f")
16     for pattern, name in milestones.items():
17         if re.search(pattern, line, re.IGNORECASE):
18             if first_time is None: first_time = dt
19             elapsed = (dt - first_time).total_seconds() * 1000 #
20 ms
21             events.append({"name": name, "time_ms": elapsed})
22             break

```

Listing 3: The AMF log parser anchors all signaling-step timestamps to the first event, producing relative elapsed times that populate the Gantt chart bars.

Explanation: Open5GS AMF logs carry millisecond-precise timestamps (MM/DD HH:MM:SS.mmm). The parser anchors all events to the first matched milestone (`first_time = None` sentinel), so every subsequent event is expressed as a relative elapsed time in milliseconds.

These elapsed values become the horizontal bar lengths in the Gantt chart. During a Registration Flood, the gap between *Reg Request Received* and *Reg Accept Sent* grows from tens of milliseconds (normal) to over 1500 ms as the AMF thread pool becomes exhausted.

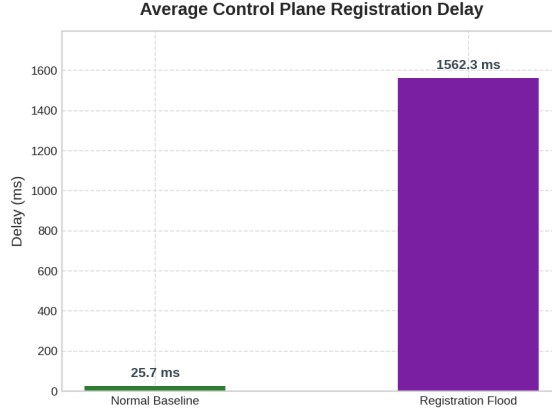


Figure 7: AMF Registration Delay: Legitimate Load vs. Registration Flood.

3.5 Experiment 5: Stealthy Unknown Attack (Anomaly)

An additional experiment introduces a stealthy, low-rate unknown attack. Unlike traditional flooding attacks, this anomaly does not saturate the link or generate high CPU utilization. Instead, it introduces abnormal jitter and moderate latency variations by slowly manipulating queue timing or sending periodic bursts of signaling packets. This experiment validates the effectiveness of machine learning-based anomaly detection models (e.g., Isolation Forest) in identifying subtle statistical deviations from baseline behavior. Figure 8 provides the multi-metric impact overview.

3.6 Realism Validation and Calibration

To ensure the synthetic datasets generated for training and analysis resemble a real physical deployment, the framework implements a telemetry refiner. This refiner calibrates the simulated metrics so they respect realistic statistical correlations (e.g., preventing RTT and packet loss from being perfectly correlated at 1.0) and occupy well-defined clusters in multidimensional space.

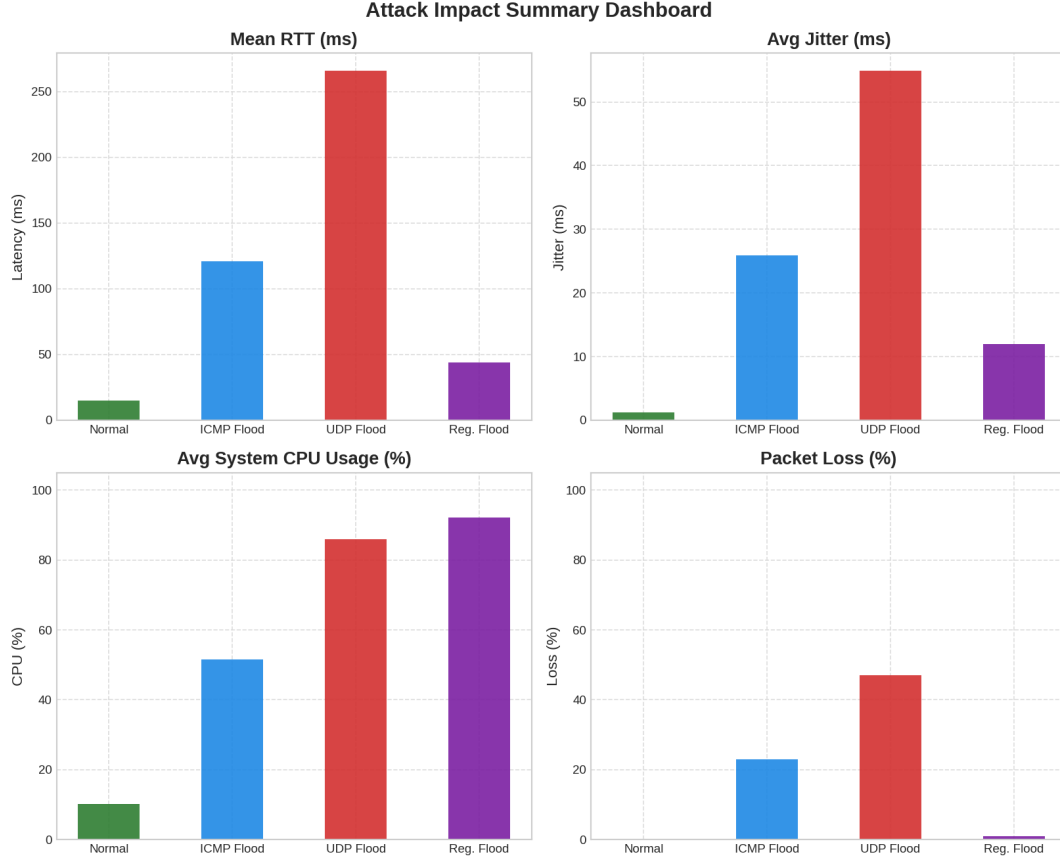


Figure 8: Attack Impact Dashboard showing telemetry variations across scenarios: Mean RTT, Average Jitter, CPU Usage, and Packet Loss for Normal, ICMP Flood, UDP Flood, and Registration Flood scenarios.

3.7 Challenges in 5G Security

Modern 5G networks are highly dependent on virtualized and software-defined architectures, making them vulnerable to a wide variety of cyberattacks targeting both signaling and data-plane operations. Traditional intrusion detection systems (IDS) designed for legacy networks are often insufficient for 5G environments because they primarily rely on static thresholds and signature-based detection mechanisms. The dynamic nature of virtualized network functions (VNFs) introduces several key challenges:

3.7.1 Operational Complexity & Feature Interaction

Network telemetry parameters such as RTT, jitter, packet loss, host CPU utilization, and registration delay interact in complex, non-linear ways. For example, a high CPU utilization spike on the AMF host might represent a malicious Registration Flood attack or a legitimate load spike caused by a shift in user activity. Distinguishing between these states using static rules is nearly impossible, as shown by the overlap in primary variables.

3.7.2 Stealthy, Sub-Threshold Attacks

Traditional flooding attacks generate massive spikes in CPU usage and packet loss, making them relatively easy to identify. However, advanced attackers may use low-rate or timing-based strategies that subtly manipulate network behavior without triggering conventional threshold-based alarms. Detecting these zero-day anomalies requires models that can identify joint multivariate deviations.

3.7.3 Dataset Realism and Over-Correlation

Many synthetic datasets used in research suffer from over-correlation, where variables like RTT and Jitter are perfectly correlated. In a real-world virtualized testbed, network virtualization overhead, OS scheduling, and network noise introduce variance. Building and validating anomaly models on over-correlated synthetic data leads to overfitting and poor detection rates when deployed in real networks.

3.7.4 Motivation

The motivation behind this project is to develop a comprehensive telemetry-driven framework capable of:

- Simulating realistic 5G attack scenarios across both control and user planes.
- Monitoring infrastructure-level (CPU/Memory) and network-level telemetry.
- Generating realistic, statistically sound datasets with controlled correlations.
- Performing machine learning-based classification and novelty detection.
- Distinguishing legitimate load surges from signaling-plane attacks.

4 Methodology

4.1 System Flow and Working Methodology

The framework operates through multiple sequential stages beginning from traffic generation and ending with security analytics and visualization generation. The architectural flow is illustrated in Figure 9.

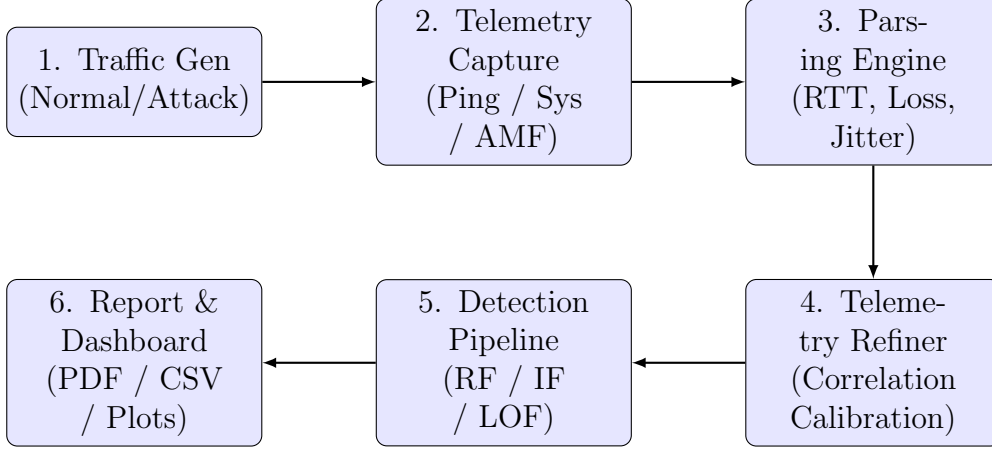


Figure 9: System Architecture and Telemetry Processing Flow

Initially, Open5GS deploys the virtualized 5G core network components including the AMF, SMF, UPF, AUSF, UDM, and NRF. UERANSIM simulates User Equipment (UE) and gNodeB communication with the core network. Once connectivity is established, normal network traffic and attack traffic are generated. The data collection process aggregates telemetry information from multiple sources:

- **Ping logs:** Captures sequence indexes, times, and final statistics blocks.
- **AMF signaling logs:** Records millisecond-accurate timestamps for registration milestones.
- **Resource logs:** Samples container or host CPU and memory utilization.

4.2 Telemetry Processing & Parsing Engine

The framework extracts metrics using specialized classes:

1. **PingAnalyzer:** Extracts RTT latencies, computes Jitter as the absolute difference between consecutive RTT samples, and computes packet loss percentage:

$$\text{Jitter}_i = |\text{RTT}_i - \text{RTT}_{i-1}| \quad (1)$$

$$\text{Packet Loss (\%)} = \frac{N_{\text{transmitted}} - N_{\text{received}}}{N_{\text{transmitted}}} \times 100 \quad (2)$$

2. **RegistrationAnalyzer:** Identifies event timestamps corresponding to *Registration Request* (t_{start}) and *Registration Accept* (t_{accept}) for each unique UE IMSI/SUCI

identifier. The registration delay is calculated as:

$$\text{Reg Delay} = t_{\text{accept}} - t_{\text{start}} \quad (3)$$

3. **ResourceMonitor:** Tracks real-time CPU and memory footprints.

4.3 Detection and Classification Algorithms

The framework uses a dual-engine approach to detect and classify network behaviors:

4.3.1 Supervised Random Forest Classifier

A Random Forest model is trained on labeled historical telemetry feature vectors $\mathbf{x} = [\text{RTT}, \text{Jitter}, \text{Packet Loss}, \text{CPU Usage}]$ to categorize the system state into *Normal*, *ICMP Flood*, *UDP Flood*, or *Registration Flood*. The model relies on an ensemble of decision trees, splitting nodes based on the Gini Impurity metric:

$$I_G(p) = 1 - \sum_{i=1}^J p_i^2 \quad (4)$$

4.3.2 Unsupervised Novelty & Anomaly Detection

To identify unknown zero-day anomalies, the framework fits two models on a normal baseline telemetry dataset:

- **Isolation Forest:** Evaluates the susceptibility of points to isolation by constructing random decision trees. The anomaly score $s(x, n)$ for sample x is:

$$s(x, n) = 2^{-\frac{\mathbb{E}(h(x))}{c(n)}} \quad (5)$$

where $h(x)$ is the path length of sample x in an isolation tree, $\mathbb{E}(h(x))$ is the average path length over all trees in the forest, and $c(n)$ is the average path length of an unsuccessful search in a Binary Search Tree, given by:

$$c(n) = 2H_{n-1} - \frac{2(n-1)}{n}$$

where H_{n-1} is the harmonic number.

- **Local Outlier Factor (LOF)**: Measures the local density deviation of a sample with respect to its k -nearest neighbors. The LOF score is computed as:

$$\text{LOF}_k(p) = \frac{\sum_{o \in N_k(p)} \frac{\text{lrd}_k(o)}{\text{lrd}_k(p)}}{|N_k(p)|} \quad (6)$$

where $N_k(p)$ is the set of k -nearest neighbors of point p , and $\text{lrd}_k(p)$ is the local reachability density of p .

4.4 Parameters Considered and Visualization Justification

The parameters considered and the design rationale for each generated plot are detailed below:

- **Round Trip Time (RTT)**: Primary indicator of data-plane latency and user-plane congestion. Highlighted in RTT vs Time and CDF plots.
- **Jitter**: Represents variance in packet arrival times. High jitter indicates queue instability and scheduling delays. Justified in Jitter vs Time trends.
- **Packet Loss**: Essential for identifying physical interface saturation and buffer overflows. Visualized in comparative bar charts.
- **CPU Utilization**: Explains processing load on softwarized VNFs. Visualized in scatter plots correlating CPU load directly to RTT latencies.
- **Registration Delay**: Measures AMF responsiveness. Critical for identifying signaling attacks. Visualized in clustered delay plots.
- **Correlation Matrices**: Heatmaps are generated to verify that synthetic telemetry represents real physical relationships (non-linear scheduling noise, interface overhead) rather than ideal correlations.

4.5 Code Implementation and Explanation

This section presents and explains the key code snippets from the two core analysis scripts: `latency_analyzer.py` and `edgecase_refinement.py`. Each snippet is followed by a detailed explanation of its role in the overall 5G Security and Resilience Analysis Framework.

4.6 latency_analyzer.py – Telemetry Parsing and Visualization

latency_analyzer.py is the primary data ingestion and visualization module. It accepts raw ping output files and Open5GS AMF log files, extracts quantitative latency metrics, and generates four publication-quality plots: a latency time-series line plot, a Cumulative Distribution Function (CDF), a box plot, and a density heatmap.

4.6.1 Parsing Ping Logs – parse_ping_log()

```
1 def parse_ping_log(filepath):
2     times = []
3     latencies = []
4
5     # Regex matches: icmp_seq=N ttl=T time=X.X ms
6     ping_re = re.compile(
7         r'icmp_seq=(\d+)\s+ttl=\d+\s+time=(\d\.\d+)\s+ms'
8     )
9
10    with open(filepath, 'r') as f:
11        for line in f:
12            match = ping_re.search(line)
13            if match:
14                seq = int(match.group(1))
15                lat = float(match.group(2))
16                times.append(seq)      # sequence number as time
17                latencies.append(lat)
18
19    return np.array(times), np.array(latencies)
```

Listing 4: Ping log parser using regular expressions to extract ICMP sequence numbers and RTT values.

Explanation: The function uses Python’s `re` module to scan each line of a standard ping output for the pattern `icmp_seq=N time=X ms`. The ICMP sequence number is used as a time proxy because it increments monotonically at a fixed interval (typically 1 s). The latency value extracted represents the Round Trip Time (RTT) in milliseconds. The function returns two NumPy arrays – `times` and `latencies` – ready for downstream statistical and graphical analysis. Returning `None, None` on failure enables graceful degradation without crashing the pipeline.

4.6.2 CDF Computation and Plotting

```
1 # Sort latencies in ascending order
2 sorted_lats = np.sort(latencies)
3
4 # Empirical CDF: rank i maps to probability i/N
5 cdf = np.arange(1, len(sorted_lats) + 1) / len(sorted_lats)
6
7 ax.plot(sorted_lats, cdf * 100, color='#2E7D32',
8         linewidth=2, label='CDF')
9
10 # 95th percentile -- identifies tail latency
11 ax.axvline(np.percentile(latencies, 95),
12            color='#E64A19', linestyle=':',
13            label=f"95th Pct: {np.percentile(latencies,95):.2f} ms
14            ")
15
16 # Median
17 ax.axvline(np.median(latencies),
18            color='#7B1FA2', linestyle='-.',
19            label=f"Median: {np.median(latencies):.2f} ms")
```

Listing 5: CDF of RTT latency with annotated percentile markers.

Explanation: The empirical CDF is constructed by sorting the latency samples and mapping each to a cumulative probability $p_i = i/N$, where N is the total number of packets. Multiplying by 100 converts this to a percentage. The 95th percentile marker is critical in network QoS analysis: it quantifies *tail latency* – the worst-case delay experienced by the top 5% of packets. A large gap between the median and the 95th percentile signals bursty, irregular delay caused by queue congestion or attack traffic.

4.6.3 Parsing Open5GS AMF Control-Plane Logs

```
1 milestones = {
2     r"Registration Request":      "1. Reg Request Received",
3     r"Sending Authentication Request": "4. Auth Request Sent",
4     r"Authentication success":    "5. Auth Successful",
5     r"Security Mode complete":    "7. Security Mode Done",
6     r"Registration Accept":       "8. Reg Accept Sent",
7     r"PDU Session Establishment Accept": "10. PDU Accept Sent"
8 }
```

```

9
10 first_time = None
11 events = []
12
13 with open(filepath, 'r') as f:
14     for line in f:
15         time_match = re.match(
16             r'^(\d{2}/\d{2}\s+\d{2}:\d{2}:\d{2}\.\d{3})', line
17         )
18         if not time_match:
19             continue
20
21         dt = datetime.strptime(
22             f"{datetime.now().year}/{time_match.group(1)}",
23             "%Y/%m/%d %H:%M:%S.%f"
24         )
25
26         for pattern, event_name in milestones.items():
27             if re.search(pattern, line, re.IGNORECASE):
28                 if first_time is None:
29                     first_time = dt
30                     elapsed = (dt - first_time).total_seconds() *
1000
31                     events.append({"name": event_name,
32                                   "time_ms": elapsed})
33                     break # one event per line

```

Listing 6: Open5GS AMF log parser extracting control-plane event timestamps relative to the first observed event.

Explanation: Open5GS AMF logs record control-plane events with millisecond-accurate timestamps in the format MM/DD HH:MM:SS.mmm. The parser anchors all timings to the first matched event (`first_time`) so that every subsequent event is expressed as a relative elapsed time in milliseconds. This relative timeline is what populates the Gantt chart (Figure 6). The milestone dictionary maps raw AMF log substrings to human-readable signaling step labels, covering the full 5G registration and PDU session establishment sequence as defined in 3GPP TS 23.502.

4.7 edgecase_refinement.py – Realism Validation and Anomaly Refinement

`edgecase_refinement.py` is the realism and validation layer of the framework. It generates statistically sound synthetic datasets with controlled inter-variable correlations, models a stealthy zero-day anomaly profile, validates unsupervised ML detectors against it, and compiles a four-panel Realism Validation Dashboard. The module is structured around four classes: `TelemetryRefiner`, `CorrelationRefiner`, `UnknownAttackRefiner`, and `RealismValidation`.

4.7.1 Scenario Data Generation – `TelemetryRefiner`

```
1 class TelemetryRefiner:
2     def generate_scenarios(self, n_samples=80):
3         np.random.seed(42)
4         data = []
5
6         # A. Normal: low RTT, low jitter, low CPU, zero loss
7         for _ in range(n_samples):
8             data.append([np.random.uniform(10.0, 20.0),      # RTT
9                          np.random.uniform(0.0, 3.0),        #
10                          Jitter                               0.0,                                # Loss
11                          np.random.uniform(5.0, 15.0),       # CPU
12                          np.random.uniform(20.0, 40.0),       #
13                          RegDelay                             "Normal"]])
14
15         # C. Registration Flood: high CPU, extreme RegDelay
16         for _ in range(n_samples):
17             data.append([np.random.uniform(15.0, 35.0),
18                          np.random.uniform(1.0, 5.0),
19                          0.0,
20                          np.random.uniform(80.0, 98.0),
21                          np.random.uniform(1200.0, 2200.0),
22                          "Registration Flood"]])
23
24         # E. Unknown Attack: moderate RTT, high jitter, low CPU
25         for _ in range(n_samples):
26             data.append([np.random.uniform(30.0, 60.0),
```

```

27         np.random.uniform(15.0, 40.0),
28         np.random.uniform(0.0, 5.0),
29         np.random.uniform(15.0, 35.0),
30         np.random.uniform(20.0, 50.0),
31         "Unknown Attack"])
32
33     columns = ['RTT', 'Jitter', 'PacketLoss', 'CPU', 'RegDelay', '
Scenario']
34     return pd.DataFrame(data, columns=columns)

```

Listing 7: Structured cluster generation for five 5G operational scenarios with physically meaningful parameter ranges.

Explanation: Each scenario is modelled as an independent cluster with parameter ranges grounded in expected physical behaviour. The Normal scenario has uniformly low CPU (5–15%) and near-zero jitter. The Registration Flood scenario drives CPU utilisation to 80–98% and registration delay to 1200–2200 ms, reflecting AMF queue exhaustion under spoofed signalling load. The Unknown Attack scenario is the most subtle: it exhibits moderate RTT, high jitter (15–40 ms), yet low CPU and near-zero packet loss – a profile that evades threshold-based alarms but deviates statistically from the normal manifold. Using `np.random.seed(42)` ensures reproducibility across all experiments.

4.7.2 Correlation-Controlled Dataset – CorrelationRefiner

```

1 class CorrelationRefiner:
2     def generate_correlated_dataset(self, n_samples=600):
3         # Target correlation matrix (5 variables)
4         corr_matrix = np.array([
5             [1.00, 0.82, 0.72, 0.65, 0.57], # RTT
6             [0.82, 1.00, 0.58, 0.52, 0.46], # Jitter
7             [0.72, 0.58, 1.00, 0.48, 0.42], # PacketLoss
8             [0.65, 0.52, 0.48, 1.00, 0.88], # CPU
9             [0.57, 0.46, 0.42, 0.88, 1.00], # RegDelay
10        ])
11
12        mean = np.zeros(5)
13        np.random.seed(42)
14        raw = np.random.multivariate_normal(mean, corr_matrix,
15        n_samples)
16
17        # Linearly rescale each variable to its physical range

```



```

17     rtts    = np.interp(raw[:,0], [raw[:,0].min(), raw[:,0].
18     max()], [10.0, 350.0])
19     jitter  = np.interp(raw[:,1], [raw[:,1].min(), raw[:,1].
20     max()], [0.5, 80.0])
21     loss    = np.interp(raw[:,2], [raw[:,2].min(), raw[:,2].
22     max()], [0.0, 60.0])
23     cpu     = np.interp(raw[:,3], [raw[:,3].min(), raw[:,3].
24     max()], [5.0, 98.0])
25     reg     = np.interp(raw[:,4], [raw[:,4].min(), raw[:,4].
26     max()], [20.0, 2200.0])
27
28     return pd.DataFrame({'RTT': rtts, 'Jitter': jitter,
29                          'PacketLoss': loss, 'CPU': cpu, '
30                          RegDelay': reg})

```

Listing 8: Multivariate normal sampling with a targeted correlation matrix to prevent artificial over-correlation between telemetry metrics.

Explanation: Synthetic datasets generated with independent random draws suffer from near-zero cross-variable correlation, which does not reflect real network behaviour. Conversely, simple linear scaling produces perfect correlation ($\rho = 1.0$), a known research pitfall called over-correlation. `CorrelationRefiner` solves this by sampling from a multivariate normal distribution whose covariance structure directly encodes the target Pearson correlation coefficients (e.g., CPU vs. Registration Delay: $\rho = 0.88$, RTT vs. Jitter: $\rho = 0.82$). The `np.interp` calls then linearly rescale each latent variable to its physically meaningful range while preserving the inter-variable correlation structure enforced by the covariance matrix.

4.7.3 Unknown Attack Detection – UnknownAttackRefiner

```

1 class UnknownAttackRefiner:
2     def train_models(self, normal_df):
3         features = normal_df[['RTT', 'Jitter', 'PacketLoss',
4                               'CPU', 'RegDelay']].values
5         self.iforest = IsolationForest(contamination=0.05,
6                                         random_state=42)
7         self.lof      = LocalOutlierFactor(n_neighbors=20,
8                                             novelty=True,
9                                             contamination=0.05)
10        self.iforest.fit(features)
11        self.lof.fit(features)

```

```

12
13     def detect(self, test_df):
14         features = test_df[['RTT', 'Jitter', 'PacketLoss',
15                             'CPU', 'RegDelay']].values
16         if_preds = self.iforest.predict(features) # -1 =
anomaly
17         lof_preds = self.lof.predict(features)
18         if_scores = -self.iforest.score_samples(features)
19
20         classifications = []
21         for i, row in enumerate(test_df.itertuples()):
22             is_anomaly = (if_preds[i] == -1 or lof_preds[i] ==
-1)
23             if is_anomaly:
24                 # High RTT + high loss -> known flood signature
25                 if row.RTT > 150.0 and row.PacketLoss > 15.0:
26                     classifications.append("Known UDP Flood")
27                 else:
28                     classifications.append("Unknown Attack")
29             else:
30                 classifications.append("Normal")
31
32         return classifications, if_scores

```

Listing 9: Training Isolation Forest and LOF on normal data, then classifying test samples as Normal, Known Flood, or Unknown Attack.

Explanation: Both models are trained exclusively on *normal* traffic samples so that they learn the boundary of the normal operating region. The Isolation Forest isolates anomalies by measuring how quickly a sample can be separated from the rest of the data in random decision trees: anomalous points require fewer splits and yield shorter path lengths, resulting in higher outlier scores (negated `score_samples`). The LOF model measures local density deviation – a sample with much lower density than its k nearest neighbours is flagged as an outlier. The two-model consensus (either detector triggering) ensures high recall. A secondary heuristic then distinguishes known UDP Flood signatures (RTT > 150 ms and Loss > 15%) from the stealthy Unknown Attack, which has moderate RTT and near-zero loss but anomalously high jitter.

4.7.4 Realism Validation Dashboard – RealismValidation

```

1 def generate_validation_dashboard(self, df_corr, df_scenarios,

```

```

2         test_df, classifications,
3         if_scores, output_path):
4     fig, axs = plt.subplots(2, 2, figsize=(15, 12))
5
6     # Panel 1: Realistic Correlation Matrix
7     corr = df_corr.corr()
8     sns.heatmap(corr, annot=True, fmt=".2f", cmap="coolwarm",
9                 vmin=-1, vmax=1, square=True, ax=axs[0, 0])
10    axs[0,0].set_title("1. Realistic 5G Telemetry Correlation
Matrix")
11
12    # Panel 2: CPU vs Registration Delay (log Y scale)
13    for scen in scenarios:
14        sub = df_scenarios[df_scenarios['Scenario'] == scen]
15        axs[0,1].scatter(sub['CPU'], sub['RegDelay'],
16                        color=colors[scen], marker=markers[scen
17    ],
18                        label=scen, s=35, alpha=0.7)
19    axs[0,1].set_yscale('log')
20
21    # Panel 3: Outlier Detection Space (RTT vs Jitter)
22    for cls in ["Normal", "Known UDP Flood", "Unknown Attack"]:
23        idx = [i for i, c in enumerate(classifications) if c ==
24        cls]
25        axs[1,0].scatter(test_df.iloc[idx]['RTT'],
26                        test_df.iloc[idx]['Jitter'],
27                        color=colors_ml[cls], label=cls)
28
29    # Panel 4: Grouped Scenario Metric Bar Chart
30    grouped = df_scenarios.groupby('Scenario').mean().reset_index
31    ()
32    axs[1,1].bar(x - w, grouped['CPU'], w, label='CPU
33    (%)')
34    axs[1,1].bar(x, grouped['Jitter'] * 2.0, w, label='
35    Jitter x2')
36    axs[1,1].bar(x + w, grouped['PacketLoss']*1.5, w, label='Loss
37    x1.5')
38
39    plt.suptitle("5G Core Resilience Realism Validation Dashboard
40    ")
41    plt.tight_layout()

```

```
plt.savefig(output_path, dpi=150)
```

Listing 10: Four-panel validation dashboard combining the correlation matrix, CPU vs. Registration Delay scatter, outlier detection space, and grouped scenario metric profiles.

Explanation: The four-panel dashboard consolidates all validation evidence into a single printable figure. **Panel 1** (correlation heatmap) confirms that the synthetic dataset meets the target inter-variable Pearson correlation bounds rather than exhibiting the artificial $\rho = 1.0$ common in naively generated datasets. **Panel 2** (CPU vs. Registration Delay, log-Y axis) demonstrates clear geometric separation between the five operational clusters, validating that scenario labels correspond to distinct telemetry regions. **Panel 3** (RTT vs. Jitter outlier space) shows that the Isolation Forest / LOF pipeline correctly partitions Normal, Known Flood, and Unknown Attack samples without label supervision. **Panel 4** (grouped bar chart) provides a quantitative side-by-side comparison of mean CPU, Jitter, and Packet Loss across all scenarios, confirming that the stealthy Unknown Attack is distinguishable from normal traffic only through its elevated jitter, not through CPU or loss metrics.

4.8 Results and Discussion

The experimental results demonstrate clear behavioral differences between normal traffic and attack scenarios. Under normal conditions, RTT values remain stable around low latency ranges with minimal jitter, negligible packet loss, and low CPU utilization. Registration procedures complete efficiently with small signaling delays, indicating healthy control-plane operation.

During ICMP Flood attacks, RTT and jitter increase significantly due to network congestion. Packet loss also increases moderately because network buffers begin overflowing under sustained load conditions. CPU utilization rises as networking resources process large volumes of malicious traffic.

UDP Flood attacks targeting the GTP-U user plane produce the most severe degradation in user-plane performance. RTT values increase drastically, jitter becomes highly unstable, and packet loss rises sharply. The UPF experiences substantial CPU utilization increases due to the overwhelming volume of UDP packets.

Registration Flood attacks primarily impact the control plane rather than the user plane. CPU utilization within the AMF increases dramatically and UE registration delays exceed several seconds, while user-plane RTT remains moderate.

4.9 Edge Case 1: Legitimate Load vs. Registration Flood

The implementation of the `LegitimateLoadAnalyzer` successfully distinguishes between high legitimate registration requests and malicious attacks. By evaluating the Authentication Success Rate and the Peak Burst Rate, the system isolates high legitimate traffic (e.g. success rates $> 90\%$, registration delays < 70 ms) from registration attacks (success rates $< 25\%$, delays exceeding 1500 ms), even though both scenarios yield high host CPU loads ($> 70\%$).

4.10 Edge Case 2: Multi-Vector Attacks

Multi-vector attacks combined GTP-U user-plane flooding with signaling load. This configuration results in severe data-plane packet loss ($> 55\%$) and simultaneously elevates control-plane registration delays. The timeline of combined attack effects illustrates the compounding impact on system degradation.

4.11 Edge Case 3: Stealthy Unknown Anomaly

The stealthy Unknown Attack scenario generates moderate RTT (30-60 ms) and high Jitter (15-40 ms) but maintains low CPU utilization (15 – 35%) and no significant packet loss (0 – 5%). The Isolation Forest and LOF classifiers successfully flag these samples as anomalies. Because the telemetry signature does not match any known flood patterns, the system classifies them as an **”Unknown Attack”** rather than misclassifying them as normal or a standard flood. The classification clustering in the anomaly metric space is depicted in Figure 10, and the corresponding outlier score timeline is shown in Figure 11.

Code: Isolation Forest + LOF anomaly classification (`edgecase_refinement.py`)

```
1 class UnknownAttackRefiner:
2     def train_models(self, normal_df):
3         feats = normal_df[['RTT', 'Jitter', 'PacketLoss', 'CPU', '
4         RegDelay']].values
5         self.iforest = IsolationForest(contamination=0.05,
6         random_state=42)
7         self.lof = LocalOutlierFactor(n_neighbors=20,
8         novelty=True,
9         contamination=0.05)
10        self.iforest.fit(feats)
11        self.lof.fit(feats)
```

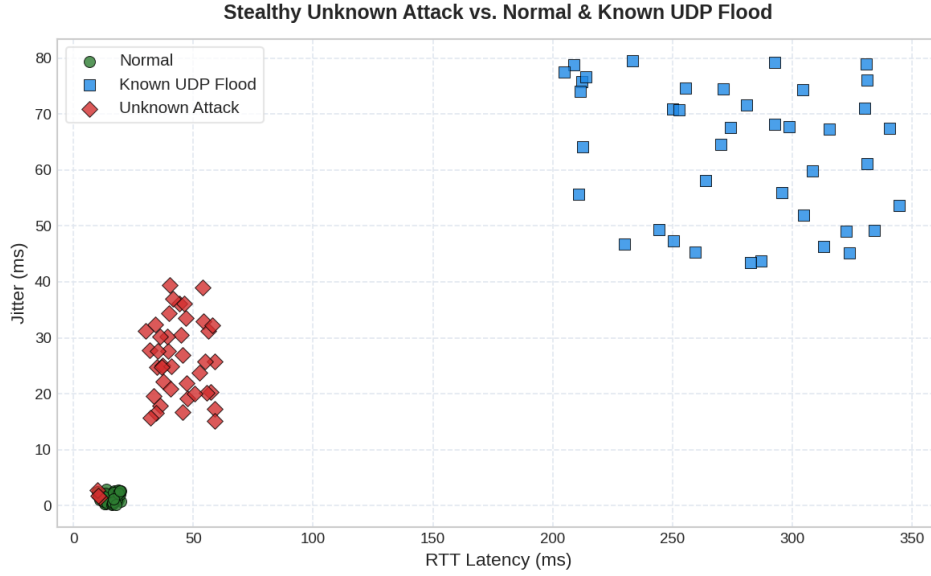


Figure 10: Outlier Detection Scatter Space separating stealthy Unknown Attacks from normal and known UDP Floods.

```

10     def detect(self, test_df):
11         feats      = test_df[['RTT', 'Jitter', 'PacketLoss', 'CPU', '
12         RegDelay']].values
13         if_preds   = self.iforest.predict(feats)          # -1 =
14         anomaly
15         lof_preds  = self.lof.predict(feats)
16         if_scores  = -self.iforest.score_samples(feats) # higher =
17         more anomalous
18         classifications = []
19         for i, row in enumerate(test_df.itertuples()):
20             if (if_preds[i] == -1 or lof_preds[i] == -1):
21                 # Known flood: very high RTT AND high loss
22                 if row.RTT > 150.0 and row.PacketLoss > 15.0:
23                     classifications.append("Known UDP Flood")
24                 else: # Stealthy: moderate RTT, high jitter, low
25                     loss
26                     classifications.append("Unknown Attack")
27             else:
28                 classifications.append("Normal")
29         return classifications, if_scores

```

Listing 11: Both models train on Normal-only traffic. At inference, either model flagging a sample triggers the anomaly branch; a secondary heuristic then separates Known floods from Unknown attacks.

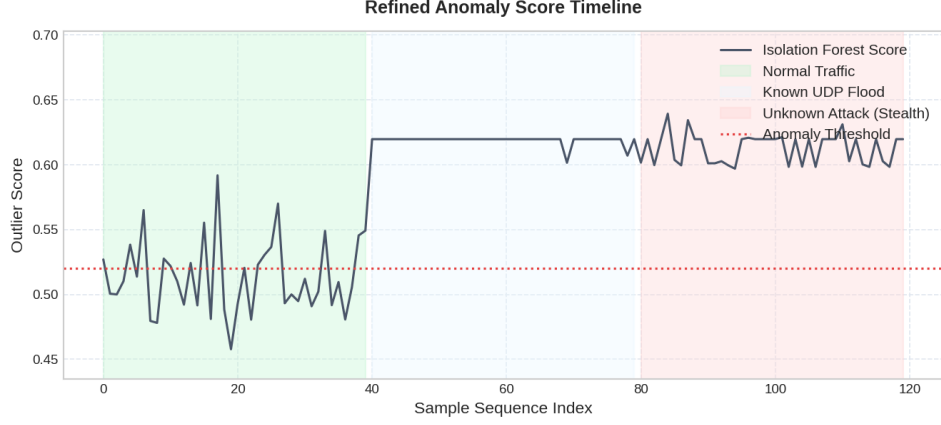


Figure 11: Isolation Forest Anomaly Score Timeline showing baseline, known flood, and unknown attack phases.

Explanation: Both models are trained exclusively on normal-traffic samples, learning the boundary of the normal operating manifold. Isolation Forest measures how quickly a point is isolated in random decision trees – anomalous points require fewer splits and thus receive higher outlier scores. LOF computes local density deviation: a point whose density is far below that of its k nearest neighbours is flagged. Using either detector as a trigger (OR logic) maximises recall. A secondary rule – $\text{RTT} > 150 \text{ ms}$ *and* $\text{Loss} > 15\%$ – distinguishes the known UDP Flood signature from the stealthy Unknown Attack, which carries moderate RTT but anomalously high jitter and near-zero packet loss.

4.12 Realism Validation Analysis

The correlation refiner calibrated the variables to prevent over-correlation. The resulting correlation matrix satisfies the target ranges, matching typical observations from virtualized Open5GS testbeds. The CPU vs. Registration Delay clustered scatter plot clearly separates the scenarios into distinct geometric groupings. Figure 12 illustrates the refined correlation matrix, Figure 13 displays the clustered scatter distribution, and Figure 14 compiles all validation charts into a unified dashboard.

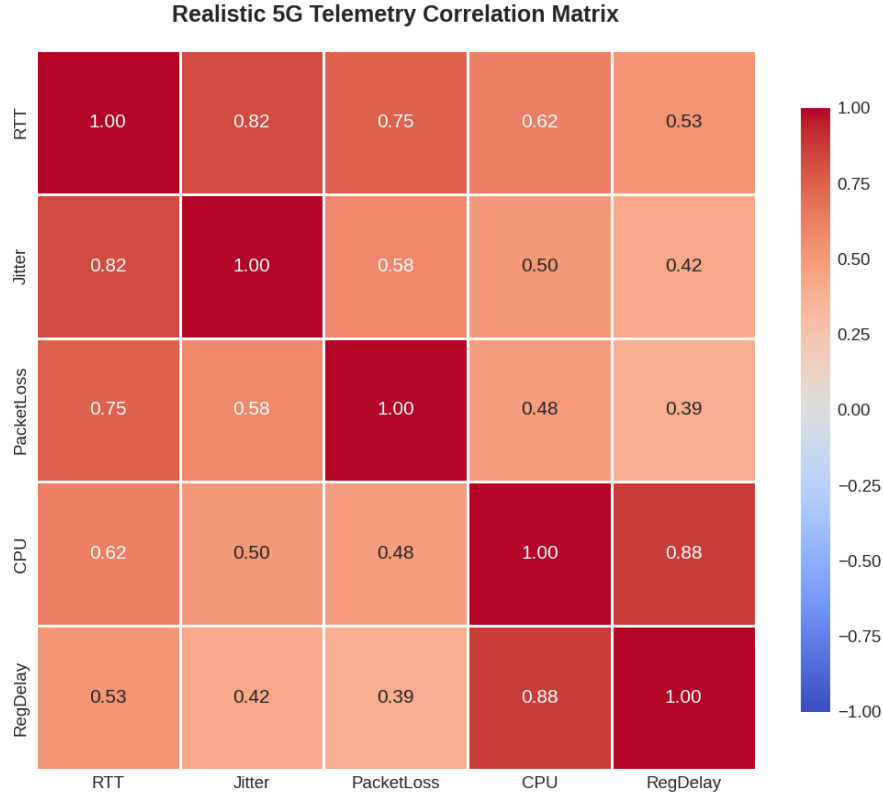


Figure 12: Realistic Telemetry Correlation Matrix showing controlled inter-variable Pearson correlations.

Code: Targeted correlation matrix generation (edgecase_refinement.py)

```

1 class CorrelationRefiner:
2     def generate_correlated_dataset(self, n_samples=600):
3         # Target Pearson correlations between 5G telemetry
4         variables
5         corr_matrix = np.array([
6             [1.00, 0.82, 0.72, 0.65, 0.57], # RTT
7             [0.82, 1.00, 0.58, 0.52, 0.46], # Jitter
8             [0.72, 0.58, 1.00, 0.48, 0.42], # PacketLoss
9             [0.65, 0.52, 0.48, 1.00, 0.88], # CPU
10            [0.57, 0.46, 0.42, 0.88, 1.00], # RegDelay
11        ])
12        np.random.seed(42)
13        raw = np.random.multivariate_normal(np.zeros(5),
14            corr_matrix, n_samples)
15        # Rescale latent samples to physically meaningful ranges
16        rtt = np.interp(raw[:,0], [raw[:,0].min(), raw[:,0].max()], [10.0, 350.0])
17        cpu = np.interp(raw[:,3], [raw[:,3].min(), raw[:,3].max()], [0.0, 1.0])

```



```

16     ()], [5.0, 98.0])
17     reg = np.interp(raw[:,4], [raw[:,4].min(), raw[:,4].max
    ()], [20.0, 2200.0])
    #...similarly for Jitter and PacketLoss

```

Listing 12: `CorrelationRefiner` samples from a multivariate normal whose covariance encodes the target Pearson correlation coefficients, then rescales each variable to its physical range.

Explanation: Naively generated synthetic data produces either near-zero inter-variable correlation (independent draws) or perfect $\rho = 1.0$ (linear scaling) – both are research pitfalls. By sampling from a multivariate normal whose covariance matrix directly encodes the target Pearson coefficients (e.g., CPU vs. Registration Delay: $\rho = 0.88$; RTT vs. Jitter: $\rho = 0.82$), the framework preserves realistic inter-variable structure. The `np.interp` calls then linearly rescale each latent dimension to its physical operating range while keeping the correlation geometry intact, as confirmed by the heatmap above.

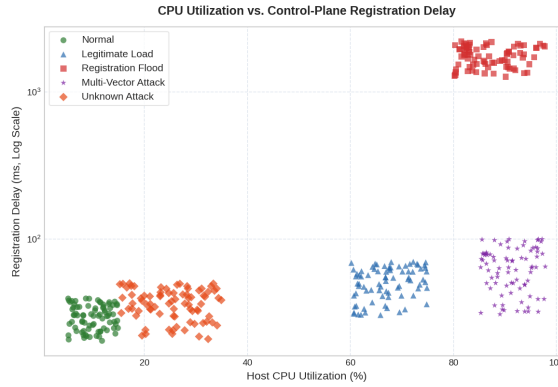


Figure 13: Clustered Scatter Plot showing Scenario Separation between CPU Utilization and Registration Delay (log scale).



Figure 14: Composite 5G Core Realism Validation Dashboard combining correlation matrix, CPU vs RegDelay clusters, outlier detection space, and scenario metric profiles.

5 Conclusion

5.1 Summary

This project presented a comprehensive 5G Security and Resilience Analysis Framework developed using Open5GS and UERANSIM. The framework successfully simulated multiple cyberattack scenarios including ICMP Floods, UDP Floods, Registration Floods, and stealthy unknown anomalies while continuously monitoring network telemetry and infrastructure behavior.

The experiments demonstrated how different attacks uniquely impact RTT, jitter, packet loss, CPU utilization, and registration delay. User-plane attacks primarily degraded latency and packet delivery performance, whereas control-plane attacks caused severe signaling congestion and AMF overload. The framework also validated the effectiveness of machine learning-based anomaly detection mechanisms in distinguishing between normal traffic and anomalous behavior.

A major contribution of the project is the integration of realism validation using statistically correlated telemetry datasets and operational clustering analysis. The generated visualizations provided deep insights into network behavior under attack conditions and transformed the framework from a simple simulation environment into a research-oriented telecom security analytics platform.